Задание 7

OpenMP. Замки (locks).

Разберите примеры 28a и 29a. Выполните задание, представленное в конце данного файла.

В OpenMP для управления доступом к критическим секциям и синхронизации потоков можно использовать более явные механизмы блокировки, такие как **замки** (locks). Для этого OpenMP предоставляет набор функций для установки и снятия блокировок, а также инициализации и уничтожения замков.

Основные функции для работы с замками:

1. omp_init_lock — инициализация замка.
2. omp_destroy_lock — уничтожение замка.
3. omp_set_lock — установка замка (блокировка).
4. omp_unset_lock — снятие замка (разблокировка).
5. omp_test_lock — попытка захватить замок без ожидания.

Структура и описание

1. **omp_set_lock(omp_lock_t *lock)** — эта функция устанавливает (захватывает) замок. Если другой поток уже захватил замок, текущий поток будет заблокирован и будет ждать до тех пор, пока замок не будет освобожден. Как только замок освобождается, текущий поток захватывает его и продолжает выполнение кода, который находится под защитой замка.
2. **omp_unset_lock(omp_lock_t *lock)** — эта функция освобождает (снимает) замок. После снятия замка его могут захватить другие потоки.

Пример использования:

```
#include <stdio.h>
#include <omp.h>

int main() {
   int sum = 0;
   omp_lock_t lock;
   // Инициализация замка. Перед тем как использовать замок, его необходимо инициализировать.
   omp_init_lock(&lock);

   #pragma omp parallel num_threads(4)
   {
   #pragma omp for
      for (int i = 0; i < 1000; ++i)
      {
         // Устанавливаем (захватываем) замок перед изменением переменной sum
         omp_set_lock(&lock);
         sum += 1;
         // Снимаем замок после изменения переменной sum
         omp_unset_lock(&lock);
      }
   }

   // Уничтожение замка. После завершения работы с замком его необходимо уничтожить для освобождения ресурсов.
   omp_destroy_lock(&lock);

   printf("Sum = %d\n", sum);
   return 0;
}
```

Функция omp_test_lock

Если вам нужно захватить замок без ожидания (т.е., если замок уже захвачен другим потоком, поток не будет ждать его освобождения, а просто продолжит выполнение кода), можно использовать функцию **omp_test_lock**.

- **omp_test_lock(omp_lock_t *lock)**: возвращает **1**, если замок был успешно захвачен, и **0**, если замок уже был захвачен другим потоком.

Пример использования omp_test_lock:

```
#include <stdio.h>
#include <omp.h>

int main() {
   int sum = 0;
   omp_lock_t lock;

   omp_init_lock(&lock);

   #pragma omp parallel num_threads(4)
   {
   #pragma omp for
      for (int i = 0; i < 1000; ++i) {
         if (omp_test_lock(&lock)) {
            sum += 1;
            omp_unset_lock(&lock);  // Освобождаем замок, если он был захвачен
         }
         // Если замок не был захвачен, поток продолжает выполнение другого кода
      }
   }

   omp_destroy_lock(&lock);

   printf("Sum = %d\n", sum);
   return 0;
}
```

Задания на использование замков (omp_set_lock и omp_unset_lock) в OpenMP.

Номер задания взять у преподавателя.

1. **Суммирование элементов массива**: Создайте массив из 10 000 элементов. Разделите его на части между потоками и выполните параллельное суммирование, используя замок для защиты общей переменной суммы.

2. **Подсчет количества четных чисел**: Параллельно проверьте все элементы массива на четность. Используйте замок для увеличения счетчика количества четных чисел.

3. **Поиск максимума в массиве**: Найдите максимальный элемент в массиве параллельно, используя замок для обновления переменной максимума.

4. **Параллельное добавление элементов в общий список**: Каждый поток генерирует несколько случайных чисел и добавляет их в общий динамический список (например, std::vector в C++). Защитите доступ к списку с помощью замков.

5. **Параллельное вычисление факториалов нескольких чисел**: Пусть каждый поток вычисляет факториал одного числа. Защитите доступ к общей переменной, которая хранит промежуточные результаты вычислений.

6. **Обратное суммирование чисел**: Пусть потоки суммируют числа в обратном порядке (например, от конца массива к началу), используя замок для защиты счетчика.

7. **Подсчет среднего значения с помощью замка**: Вычислите среднее значение элементов массива параллельно, используя замок для защиты переменной суммы и количества элементов.

8. **Параллельное сканирование массива на наличие простых чисел**: Проверьте элементы массива на простоту параллельно и подсчитайте количество простых чисел. Используйте замок для защиты счетчика.

9. **Запись в файл из нескольких потоков**: Несколько потоков записывают результаты своих вычислений в один общий файл. Используйте замок для защиты операций записи.

10. **Круговая обработка массива**: Пусть потоки обрабатывают массив циклически, каждый поток берет элемент с конца, обрабатывает его и записывает результат. Защитите доступ к индексу массива с помощью замка.

11. **Синхронизация доступа к двум переменным**: Потоки параллельно обновляют две глобальные переменные. Используйте два замка для синхронизации доступа к каждой переменной.

12. **Обновление глобального счетчика с условием**: Пусть потоки увеличивают счетчик только тогда, когда встречают определенное условие (например, элемент массива больше заданного порога). Защитите счетчик с помощью замка.

13. **Реализация параллельной очереди**: Потоки добавляют и удаляют элементы из очереди. Используйте замок для защиты операций добавления и удаления.

14. **Параллельный поиск элемента**: Найдите заданный элемент в массиве параллельно. Если поток находит элемент, он обновляет флаг в защищенной замком переменной, сигнализируя другим потокам об остановке.

15. **Реализация параллельного обмена данными между потоками**: Пусть потоки обмениваются данными друг с другом через общий буфер. Используйте замки для синхронизации доступа к буферу.

16. **Многопоточная реализация банковского счета**: Смоделируйте систему банковского счета, где несколько потоков одновременно вносят деньги на счет и снимают их. Используйте замок для защиты операций депозита и снятия средств.